

■ Pandas Master Handbook – Part 1 (Lessons 1–10)

Beginner to Data Science Level Notes

Goal of Part 1: Build a strong foundation of Pandas. By the end of these 10 lessons, you'll be able to create DataFrames, read CSV files, access data, and inspect datasets confidently.

Lesson 1: Introduction to Pandas

1. What is Pandas?

Pandas is an open-source Python library used for working with structured data.

Think of it as Excel inside Python.

With Pandas you can:

- Read CSV, Excel, SQL files
- Clean messy datasets
- Filter data
- Sort records
- Analyse information
- Prepare data for Machine Learning

It is one of the most important libraries in Data Science.

2. Why is Pandas so popular?

Without Pandas:

```
marks = [90, 80, 70]
```

```
names = ["Ram", "Shyam", "Mohan"]
```

Managing thousands of records becomes difficult.

With Pandas:

```
import pandas as pd
```

```
student = pd.DataFrame({  
    "Name": ["Ram", "Shyam", "Mohan"],  
    "Marks": [90, 80, 70]  
})
```

Everything is organised into rows and columns.

3. Installing Pandas

```
pip install pandas
```

Check installation:

```
import pandas as pd
```

```
print(pd.__version__)
```

Example output:

2.3.1

4. Importing Pandas

Always write:

```
import pandas as pd
```

Why pd?

Instead of writing:

```
pandas.DataFrame()
```

we simply write

```
pd.DataFrame()
```

It is the worldwide convention.

5. Real-world Uses

Pandas is used in:

- Netflix
 - Google
 - Amazon
 - Microsoft
 - Facebook
 - Data Science
 - AI
 - Finance
 - Healthcare
 - Research
-

6. Common Mistakes

✗

```
import Pandas
```

Python is case-sensitive.

Correct:

```
import pandas as pd
```

Interview Questions

Q1. What is Pandas?

A Python library used for data analysis and manipulation.

Q2. Why do we use pd?

It is simply an alias to make typing easier.

Memory Trick

Pandas = Excel + Python

Lesson 2: Series

What is a Series?

A Series is a one-dimensional labelled array.

Think of it as one Excel column.

Example:

Roll

1

2

3

That single column is a Series.

Syntax

`pd.Series(data)`

Example

```
import pandas as pd
```

```
numbers = pd.Series([10,20,30,40])
```

```
print(numbers)
```

Output

```
0    10
```

```
1    20
```

```
2    30
```

```
3    40
```

```
dtype: int64
```

Index

Every Series has an index.

```
0
```

```
1
```

```
2
```

```
3
```

You can also create custom indices.

```
pd.Series(
```

```
[90,80,70],  
index=["Ram", "Shyam", "Mohan"]  
)
```

Output

Ram	90
Shyam	80
Mohan	70

Accessing Values

```
numbers[0]
```

Output

10

Common Mistakes

✗

```
pd.series()
```

Correct

```
pd.Series()
```

Capital S

Interview Questions

Difference between list and Series?

List

- No labels

Series

- Has labels (index)
-

Memory Trick

Series = Single Column

Lesson 3: DataFrame

What is a DataFrame?

A DataFrame is a two-dimensional table.

Imagine Excel.

Name	Age	City
------	-----	------

Ram	20	Delhi
-----	----	-------

Shyam	21	Mumbai
-------	----	--------

```
Name Age City
Mohan 19 Kolkata
```

This entire table is a DataFrame.

Creating DataFrame

```
student = pd.DataFrame({
    "Name":["Ram", "Shyam", "Mohan"],
    "Age":[20, 21, 19],
    "City":["Delhi", "Mumbai", "Kolkata"]
})
```

Printing

```
print(student)
```

Structure

Rows

```
0
```

```
1
```

```
2
```

Columns

```
Name
```

```
Age
```

```
City
```

Data Types

Avoid

```
"Age":["20", "21", "19"]
```

These are strings.

Prefer

```
"Age":[20, 21, 19]
```

Integers.

Common Mistakes

Different column lengths.

Wrong

```
{
    "Name":["Ram", "Shyam"],
    "Age":[20]
}
```

Every column must have equal length.

Interview Questions

Difference between Series and DataFrame?

Series

- One-dimensional

DataFrame

- Two-dimensional
-

Memory Trick

Series = One Column

DataFrame = Entire Excel Sheet

Lesson 4: Selecting Columns

Suppose

```
student = pd.DataFrame({  
    "Name":["Ram", "Shyam"],  
    "Age":[20,21]  
})
```

Selecting One Column

```
student["Age"]
```

Output

```
0    20  
1    21
```

Selecting Another Column

```
student["Name"]
```

Type Returned

Selecting one column returns a Series.

Common Mistakes

Wrong

```
student[Age]
```

Correct

```
student["Age"]
```

Column names are strings.

Interview Question

Why use quotes?

Because column names are strings.

Lesson 5: Selecting Multiple Columns

Syntax

```
student[["Name", "Age"]]
```

Notice

Two square brackets.

Outer

```
[]
```

Inner

```
["Name", "Age"]
```

Output

Name	Age
Ram	20
Shyam	21

Why Double Brackets?

One bracket

↓

Series

Two brackets

↓

DataFrame

Example

```
student[["Age"]]
```

Still returns a DataFrame.

Common Mistake

Wrong

```
student["Name", "Age"]
```

Correct

```
student[["Name", "Age"]]
```

Lesson 6: loc

What is .loc?

`.loc` selects data using labels (row labels and column names).

Syntax

```
df.loc[row]
```

Example

```
student.loc[1]
```

Output

Name	Shyam
Age	21

Specific value

```
student.loc[1,"Age"]
```

Output

21

Multiple rows

```
student.loc[0:2]
```

Memory Trick

`loc` → Labels

Lesson 7: Row + Column Selection

General Syntax

```
df.loc[row,column]
```

Examples

```
student.loc[2,"City"]
```

Returns

Kolkata

Multiple columns

```
student.loc[:,["Name","Age"]]
```

Colon means

All rows.

Everything

```
student.loc[:,:]
```

Entire DataFrame.

Lesson 8: `iloc`

`iloc` uses positions.

Syntax

```
df.iloc[row,column]
```

Example

```
student.iloc[1,0]
```

Means

Second row

First column

Output

Shyam

Another example

```
student.iloc[0,2]
```

Output

Delhi

Difference

`loc`

↓

Uses labels

`iloc`

↓

Uses positions

Memory Trick

`iloc` = Integer location

Lesson 9: Reading CSV Files

CSV

Comma Separated Values

Example

Name, Age, City

Ram, 20, Delhi

Shyam, 21, Mumbai

Read CSV

```
df = pd.read_csv("students.csv")
```

Print

```
print(df)
```

Common Mistakes

Wrong file path

Wrong filename

Missing CSV

Useful Parameters

```
pd.read_csv("students.csv")
```

Later you'll learn

```
pd.read_csv(  
    "students.csv",  
    index_col=0  
)
```

Interview Question

Difference between CSV and Excel?

CSV

Simple text

Excel

Rich formatting, multiple sheets

Lesson 10: head()

Suppose dataset has

1,000,000 rows

Printing everything is impossible.

Use

```
df.head()
```

Output

First five rows.

Specific number

```
df.head(10)
```

First ten rows.

Similarly

```
df.tail()
```

Last five rows.

Why `head()`?

Quick inspection.

Most Data Scientists immediately use

`df.head()`

after reading data.

Common Mistake

Wrong

`df.Head()`

Python is case-sensitive.

Correct

`df.head()`

Part 1 Summary

Lesson	Topic	Key Syntax
1	Import Pandas	<code>import pandas as pd</code>
2	Series	<code>pd.Series()</code>
3	DataFrame	<code>pd.DataFrame()</code>
4	One Column	<code>df["Age"]</code>
5	Multiple Columns	<code>df[["Name", "Age"]]</code>
6	<code>loc</code>	<code>df.loc[row]</code>
7	<code>loc</code> Row & Column	<code>df.loc[row, column]</code>
8	<code>iloc</code>	<code>df.iloc[row, column]</code>
9	Read CSV	<code>pd.read_csv()</code>
10	Head	<code>df.head()</code>

Part 1 Interview Cheat Sheet

- Series → One-dimensional labelled array.
- DataFrame → Two-dimensional table.
- `loc` → Uses labels.
- `iloc` → Uses integer positions.
- Single `[]` → Usually returns a Series.
- Double `[[ ]]` → Returns a DataFrame.
- `head()` → First 5 rows.
- `read_csv()` → Reads CSV files.
- Always import: `import pandas as pd`

■ Pandas Master Handbook – Part 2 (Lessons 11-20)

Exploring, Filtering & Manipulating Data

Goal of Part 2: Learn how to inspect datasets, filter records, sort data, and rename columns. These are the operations you'll use almost every day in data analysis.

Lesson 11: info() – Dataset Information

1. What is info()?

After loading a dataset, one of the first things a data analyst checks is:

- How many rows are there?
- How many columns?
- What are the data types?
- Are there missing values?

The info() method answers all these questions.

Syntax

```
df.info()
```

Example

```
import pandas as pd
```

```
student = pd.DataFrame({  
    "Name": ["Ram", "Shyam", "Mohan"],  
    "Age": [20, 21, 19],  
    "City": ["Delhi", "Mumbai", "Kolkata"]  
})
```

```
student.info()
```

Output (simplified):

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 3 entries, 0 to 2
```

```
Data columns (total 3 columns):
```

```
Name      object
```

```
Age       int64
```

```
City      object
```

Understanding the Output

RangeIndex

```
RangeIndex: 3 entries, 0 to 2
```

Meaning:

- Total rows = 3
- Index starts from 0

Data columns

Data columns (total 3 columns)

Means:

The DataFrame has 3 columns.

Non-Null Count

Age 3 non-null

All values are present.

Suppose:

Age = [20, None, 19]

Output becomes

Age 2 non-null

Meaning one value is missing.

Dtype

Common data types:

Data Type	Meaning
-----------	---------

int64	Integer
-------	---------

float64	Decimal
---------	---------

object	Text
--------	------

bool	True/False
------	------------

datetime64	Date & Time
------------	-------------

Why use info()?

Imagine a dataset with

500,000 rows

80 columns

Instead of checking manually,

df.info()

gives everything instantly.

Common Mistakes

Wrong

df.Info()

Correct

df.info()

Interview Questions

Q1. What information does `info()` provide?

- Rows
- Columns
- Data types
- Missing values
- Memory usage

Q2. Does `info()` return a `DataFrame`?

No.

It prints information about the `DataFrame`.

Memory Trick

Think of `info()` as your `DataFrame`'s identity card.

Lesson 12: `describe()` – Statistical Summary

What is `describe()`?

This method gives statistical information about numeric columns.

Syntax

```
df.describe()
```

Example

```
student.describe()
```

Output

Age

count	3
mean	20
std	1
min	19
25%	19.5
50%	20
75%	20.5
max	21

Meaning of Each Statistic

count

Number of values.

mean

Average.

Formula

Sum of values

Number of values

std

Standard deviation.

Measures data spread.

min

Smallest value.

max

Largest value.

25%

First Quartile

25% of observations lie below this value.

50%

Median.

Middle value.

75%

Third Quartile.

Include Text Columns

`df.describe(include="all")`

Useful when your DataFrame contains strings.

Interview Questions

Difference between

`df.info()`

and

`df.describe()`

info() describe()

Data types Statistics

Missing values Mean

Rows Min

Columns Max

Memory Trick

describe() = Report Card of Numbers.

Lesson 13: shape

What is shape?

Returns

(Number of Rows,
 Number of Columns)

Syntax

df.shape

Output

(100,5)

Meaning

100 rows

5 columns

Important

shape

is an

ATTRIBUTE

not

METHOD

Correct

df.shape

Wrong

df.shape()

Extract Individually

Rows

df.shape[0]

Columns

df.shape[1]

Example

`student.shape`

Output

`(3,3)`

Interview Questions

Difference

shape

vs

`len(df)`

`len(df)` → Rows only

shape → Rows and Columns

Memory Trick

Shape = Size

Lesson 14: columns

What is columns?

Returns all column names.

Syntax

`df.columns`

Output

`Index(['Name', 'Age', 'City'])`

Convert to list

`list(df.columns)`

Output

`['Name', 'Age', 'City']`

Rename All Columns

`df.columns=[`

`"Name",`

`"Age",`

`"City"`

`]`

Common Mistakes

Wrong

`df.columns()`

Correct

```
df.columns
```

Memory Trick

Columns = Column Names

Lesson 15: Filtering Rows

Suppose

```
student = pd.DataFrame({  
    "Name":["Ram", "Shyam", "Mohan"],  
    "Age": [20, 21, 19]  
})
```

Students above 20

```
student[  
    student["Age"]>20  
]
```

Output

Shyam

Less than

```
student[  
    student["Age"]<20  
]
```

Equal

```
student[  
    student["Age"]==20  
]
```

Not Equal

```
student[  
    student["Age"]!=20  
]
```

Comparison Operators

Symbol Meaning

> Greater

< Smaller

Symbol Meaning

`>=` Greater Equal

`<=` Smaller Equal

`==` Equal

`!=` Not Equal

Memory Trick

Filtering returns

Only

TRUE rows.

Lesson 16: Multiple Conditions

Sometimes

One condition

isn't enough.

AND

```
student[
(student["Age"]>=20)
&
(student["Age"]<=21)
]
```

OR

```
student[
(student["Age"]<20)
|
(student["Age"]>21)
]
```

NOT

```
student[
~(student["Age"]==20)
]
```

Important

Always use parentheses.

Correct

`(A)&(B)`

Wrong

A&B

Logical Operators

Operator Meaning

& AND

| OR

~ NOT

Memory Trick

Each condition gets its own brackets.

Lesson 17: Filter + Select Columns

Sometimes

Filter rows

and

show only selected columns.

Syntax

```
student.loc[
student["Age"]>=20,
"Name"
]
```

Output

Ram

Shyam

Multiple columns

```
student.loc[
student["Age"]>=20,
["Name","City"]
]
```

Remember

```
.loc[
rows,
columns
]
```

Why use .loc?

Cleaner

Faster

Readable

Preferred in industry.

Lesson 18: Sorting Data

Ascending

```
student.sort_values("Age")
```

Descending

```
student.sort_values(
```

```
"Age",
```

```
ascending=False
```

```
)
```

Sort by Name

```
student.sort_values(
```

```
"Name"
```

```
)
```

Common Mistakes

Wrong

```
student.sort_values[]
```

Correct

```
student.sort_values()
```

Memory Trick

Sorting is a method.

Methods need

() .

Lesson 19: Multi-column Sorting

Suppose

Age	Name
-----	------

20	Ram
----	-----

20	Amit
----	------

19	Mohan
----	-------

Sort by

```
Age
then
Name
student.sort_values(
["Age", "Name"]
)
```

Different order

```
student.sort_values(
["Age", "Name"],
ascending=[False, True]
)
```

Meaning

Age ↓

Name ↑

Useful in reports

Leaderboards

Ranking systems

Payroll

Sales data

Lesson 20: Renaming Columns

Rename one column

```
student.rename(
columns={
"Name": "Student_Name"
}
)
```

Rename multiple

```
student.rename(
columns={
"Name": "Student_Name",
"Age": "Student_Age"
}
)
```

Permanent Rename

```
student.rename(
```

```
columns={
    "Name": "Student_Name"
},
inplace=True
)
```

Or

```
student.columns=[
    "Student_Name",
    "Age",
    "City"
]
```

Difference

`rename()`

Rename selected columns.

`columns`

Replace every column name.

Part 2 Summary

Lesson	Topic	Key Syntax
11	Dataset Information	<code>df.info()</code>
12	Statistical Summary	<code>df.describe()</code>
13	Shape	<code>df.shape</code>
14	Column Names	<code>df.columns</code>
15	Filtering	<code>df[df["Age"] > 20]</code>
16	Multiple Conditions (A) & (B)	
17	Filter + Select	<code>df.loc[rows, columns]</code>
18	Sort	<code>df.sort_values()</code>
19	Multi-sort	<code>sort_values(...)</code>
20	Rename	<code>rename(columns={})</code>

Part 2 Interview Cheat Sheet

Difference between `info()` and `describe()`

<code>info()</code>	<code>describe()</code>
---------------------	-------------------------

Dataset structure	Statistical summary
-------------------	---------------------

Data types	Mean, min, max, quartiles
------------	---------------------------

`info()` `describe()`
`Missing values` `Numeric analysis`

`shape`
`df.shape`
Returns:
(rows, columns)

`columns`
`df.columns`
Returns column names.

Filtering
`df[df["Marks"] > 80]`

Multiple Conditions
(A) & (B)
AND
(A) | (B)
OR
~(A)
NOT

Sorting
Ascending:
`df.sort_values("Marks")`
Descending:
`df.sort_values("Marks", ascending=False)`

Rename
`df.rename(columns={"Old": "New"})`

Pandas Master Handbook – Part 3 (Lessons 21–30)

Data Cleaning, Grouping, Merging & Real-World Data Analysis

Goal of Part 3: Learn the Pandas operations used by Data Analysts and Data Scientists every day. By the end of these lessons, you'll be able to clean datasets, combine multiple tables, analyse grouped data, and save your work.

Lesson 21: Adding a New Column

What is a New Column?

Sometimes your dataset doesn't contain all the information you need.

Example:

Name	Age
Ram	20
Shyam	21

Suppose you want to add Marks.

Syntax

```
df["Column_Name"] = values
```

Example

```
student["Marks"] = [85, 90]
```

Output

Name	Age	Marks
Ram	20	85
Shyam	21	90

Add Same Value

```
student["Country"] = "India"
```

Every row becomes

India

India

India

Create Using Existing Columns

```
student["Age_After_5_Years"] = student["Age"] + 5
```

Common Mistakes

Wrong length

```
student["Marks"] = [80]
```

If there are 3 rows, Pandas raises an error.

Interview Question

Can a new column be created using existing columns?

Yes.

```
df["Total"] = df["Math"] + df["Science"]
```

Memory Trick

= creates or updates a column.

Lesson 22: Deleting Columns

Sometimes a dataset has unnecessary columns.

Method 1

```
student.drop("Marks", axis=1)
```

Method 2 (Recommended)

```
student.drop(columns=["Marks"])
```

Multiple Columns

```
student.drop(  
columns=[  
"Age",  
"City"  
]  
)
```

Permanent Delete

```
student.drop(  
columns=["Marks"],  
inplace=True  
)
```

Difference

Without

```
inplace=True
```

Original DataFrame stays unchanged.

With

```
inplace=True
```

Original DataFrame changes permanently.

Memory Trick

Drop = Remove.

Lesson 23: Missing Values

Real datasets are rarely perfect.

Example

Name Age

Ram 20

Shyam NaN

Mohan 19

NaN means

Not a Number

It represents missing data.

Detect Missing Values

```
df.isnull()
```

Output

False

True

False

Count Missing Values

```
df.isnull().sum()
```

Remove Missing Rows

```
df.dropna()
```

Fill Missing Values

Replace with 0

```
df.fillna(0)
```

Replace with average

```
df["Age"] = df["Age"].fillna(  
df["Age"].mean()  
)
```

Why Fill Instead of Delete?

Suppose

100,000 rows

10 rows missing

Deleting all rows may remove useful data.

Replacing missing values is often better.

Interview Questions

Difference

dropna()

↓

Removes rows

`fillna()`

↓

Keeps rows

Memory Trick

NaN = Missing.

Lesson 24: Unique Values

Suppose

City

Delhi

Delhi

Mumbai

Kolkata

Find Unique Values

`df["City"].unique()`

Output

Delhi

Mumbai

Kolkata

Count Unique

`df["City"].nunique()`

Output

3

Why Useful?

Find

- Number of cities
 - Departments
 - Products
 - Categories
-

Memory Trick

Unique = No duplicates.

Lesson 25: Value Counts

Sometimes we need frequencies.

Dataset

City

Delhi

Delhi

Mumbai

Delhi

Syntax

```
df["City"].value_counts()
```

Output

Delhi 3

Mumbai 1

Why Useful?

Most sold product

Most common city

Most frequent category

Most common disease

Most watched movie

Sort Automatically

Highest frequency appears first.

Interview Question

Difference

`unique()`

↓

Returns values

`value_counts()`

↓

Returns frequencies

Lesson 26: GroupBy

One of the most powerful Pandas functions.

Suppose

City Marks

Delhi 80

Delhi 90

Mumbai 70

Average Marks

```
df.groupby("City")["Marks"].mean()
```

Output

Delhi 85

Mumbai 70

Other Functions

Maximum

```
df.groupby(  
    "City"  
)["Marks"].max()
```

Minimum

```
df.groupby(  
    "City"  
)["Marks"].min()
```

Count

```
df.groupby(  
    "City"  
)["Marks"].count()
```

Sum

```
df.groupby(  
    "City"  
)["Marks"].sum()
```

Why GroupBy?

Sales reports

Monthly revenue

Employee salary

Department analysis

Customer analytics

Machine Learning

Memory Trick

Group first

Calculate later.

Lesson 27: Merge DataFrames

Suppose

Students Table

ID Name

1 Ram

2 Shyam

Marks Table

ID Marks

1 90

2 80

Merge

```
pd.merge(  
students,  
marks,  
on="ID"  
)
```

Output

ID Name Marks

1 Ram 90

2 Shyam 80

Types

Inner

how="inner"

Common rows only.

Left

how="left"

All left rows.

Right

how="right"

All right rows.

Outer

how="outer"

Everything.

Memory Trick

Merge = SQL JOIN.

Lesson 28: Export Data

After cleaning

Save results.

CSV

```
df.to_csv(  
    "students.csv",  
    index=False  
)
```

Excel

```
df.to_excel(  
    "students.xlsx",  
    index=False  
)
```

Why index=False?

Otherwise

0

1

2

is saved as an extra column.

Interview Question

Why use index=False?

To avoid saving row numbers as another column.

Lesson 29: apply()

Very useful.

Apply a function to every value.

Example

Square Marks

```
df["Marks"] = df["Marks"].apply(  
    lambda x: x**2  
)
```

```
lambda x: x*x
```

```
)
```

Uppercase Names

```
df["Name"] = df["Name"].apply(
```

```
str.upper
```

```
)
```

Length of Name

```
df["Length"] = df["Name"].apply(
```

```
len
```

```
)
```

Why Use apply()?

Cleaning text

Formatting

Feature Engineering

Machine Learning

Interview Question

Difference

Loop

↓

Slower

apply()

↓

Cleaner

Often faster

Memory Trick

Apply = Apply something to every row or value.

Lesson 30: Mini Project

Dataset

```
import pandas as pd
```

```
student = pd.DataFrame({
```

```
    "Name":["Ram", "Shyam", "Mohan", "Amit"],
```

```
    "Age":[20, 21, 19, 22],
```

```
    "Marks":[80, 90, 75, 85],
```

```
    "City":["Delhi","Mumbai","Delhi","Kolkata"]
})
```

Step 1

See first rows

```
student.head()
```

Step 2

Dataset information

```
student.info()
```

Step 3

Statistics

```
student.describe()
```

Step 4

Shape

```
student.shape
```

Step 5

Students with Marks > 80

```
student[
student["Marks"]>80
]
```

Step 6

Sort

```
student.sort_values(
"Marks",
ascending=False
)
```

Step 7

Average Marks

```
student["Marks"].mean()
```

Step 8

Students from Delhi

```
student[
student["City"]=="Delhi"
```

]

Step 9

Average Marks by City

```
student.groupby(  
    "City"  
)["Marks"].mean()
```

Step 10

Save Results

```
student.to_csv(  
    "student.csv",  
    index=False  
)
```

Complete Pandas Cheat Sheet

Task	Syntax
Import	<code>import pandas as pd</code>
Series	<code>pd.Series()</code>
DataFrame	<code>pd.DataFrame()</code>
Read CSV	<code>pd.read_csv()</code>
First Rows	<code>df.head()</code>
Last Rows	<code>df.tail()</code>
Information	<code>df.info()</code>
Statistics	<code>df.describe()</code>
Shape	<code>df.shape</code>
Columns	<code>df.columns</code>
Select Column	<code>df["Age"]</code>
Multiple Columns	<code>df[["Age", "Marks"]]</code>
loc	<code>df.loc[]</code>
iloc	<code>df.iloc[]</code>
Filter	<code>df[df["Age"]>20]</code>
Multiple Conditions (A)&(B)	
Sort	<code>df.sort_values()</code>
Rename	<code>df.rename()</code>
Add Column	<code>df["New"]=</code>

Task	Syntax
Drop Column	<code>df.drop()</code>
Missing Values	<code>dropna()</code> , <code>fillna()</code>
Unique	<code>unique()</code>
Count	<code>value_counts()</code>
Group	<code>groupby()</code>
Merge	<code>merge()</code>
Apply	<code>apply()</code>
Save CSV	<code>to_csv()</code>
Save Excel	<code>to_excel()</code>

Top 20 Pandas Interview Questions

1. What is Pandas?
2. Difference between Series and DataFrame?
3. Difference between loc and iloc?
4. Difference between head() and tail()?
5. Difference between info() and describe()?
6. What does shape return?
7. Difference between unique() and value_counts()?
8. What is groupby()?
9. Difference between merge() and concat()?
10. What is apply()?
11. What is fillna()?
12. What is dropna()?
13. Difference between == and =?
14. How do you sort a DataFrame?
15. How do you rename columns?
16. How do you add a new column?
17. Why use index=False in to_csv()?
18. What are missing values?
19. How do you filter multiple conditions?
20. How do you read a CSV file?

Congratulations!

You've now completed a beginner-to-intermediate Pandas roadmap covering the concepts you'll encounter most often in data analysis.